



University
of Basel

Chapter 5.3: Parallel Programming – Shared Memory

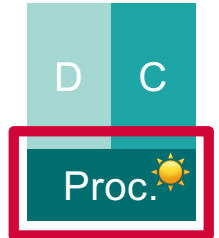
Foundations of Distributed Systems

Fall Semester 2025

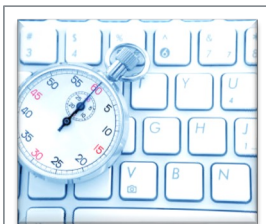
Prof. Dr. Florina Ciorba



Addressed in The **Processing** Lecture Series



• Goals



Performance

```
00111101001010101
01001010100010101
01010ERROR011010
10100010100110001
01100010100100111
10100010100110001
```

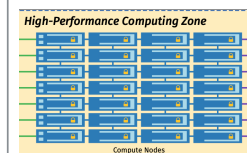
Resiliency



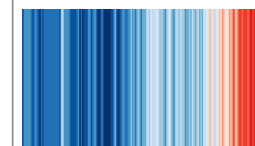
**Extreme
scalability**



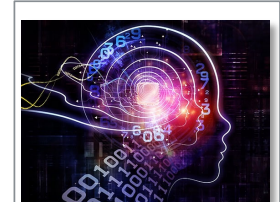
Energy



Security



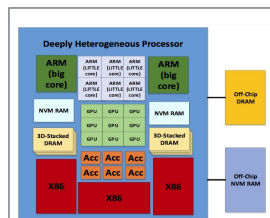
Sustainability



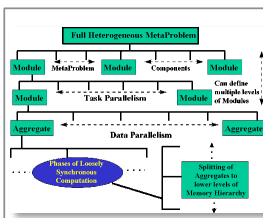
**Convergence
HPC+Data+AI**

• Challenges

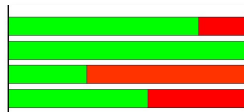
Chapter 5.3: Parallel programming



Heterogeneity



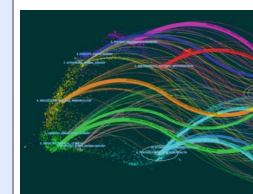
**Multilevel
parallelism**



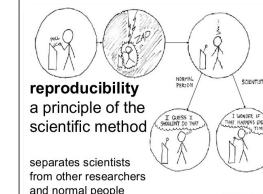
Load imbalance



Programming



Visualization



Reproducibility

Overview of Chapter 5.3: Parallel Programming

Introduction to Parallel Programming

- a Shared-Memory Programming: OpenMP Introduction
- b Distributed-Memory Programming: MPI Introduction

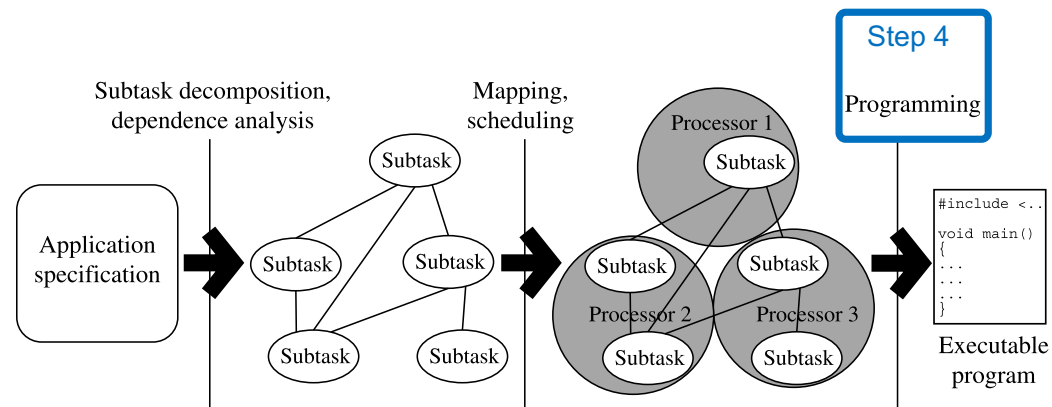


Figure 1.1. Parallel programming—process of parallelization.

Parallel Programming – How?

01

Extend compilers

Translate sequential programs into parallel programs

02

Extend languages

Add parallel *operations* on top of sequential language. A low-level approach

03

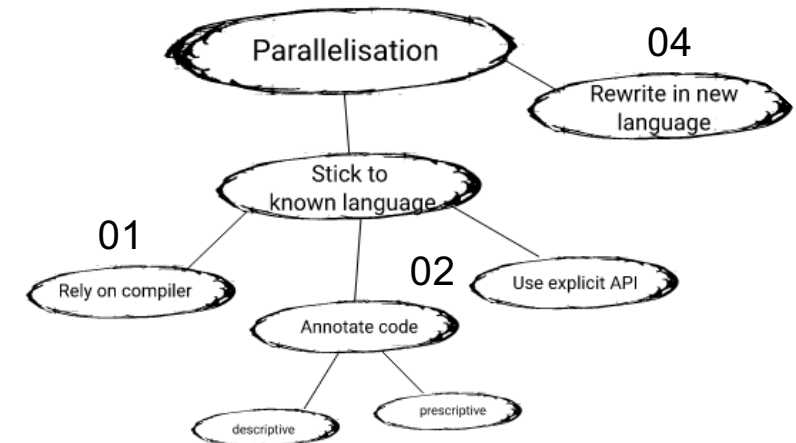
Stack languages

Add a parallel *language* layer on top of sequential language

04

New language & compiler set

Define a totally new parallel language and compiler system



Parallel Programming – Why Low-level Approach?

02

Extend languages

Add *parallel operations* on top of sequential language. A low-level approach



Pros

- Easiest, quickest, and least expensive
- Leverages existing compiler technology
- New libraries for extensions to language can be ready soon after new parallel computers are available
- No rewriting of codes in new language



Cons

- Lack of compiler support to catch errors related to management of parallelism, such as
 - Creating & terminating processes
 - Synchronizing processes
 - Communication between processes
- Easy to write programs that are difficult to understand or debug

In the early days of parallel computing we were obsessed with finding the “right” parallel programming environment

ABCPL	CORRELATE	GLU	Mentat	Parafrese2	
ACE	CPS	GUARD	Legion	Paralation	pC++
ACT++	CRL	HASL	Meta Chaos	Parallel-C++	SCHEDULE
Active messages	CSP	Haskell	Midway	Parallaxis	SciTL
Adl	Cthreads	HPC++	Millipede	ParC	POET
Adsmith	CUMULVS	JAVAR.	CparPar	ParLib++	SDDA.
ADDAP	DAGGER	HORUS	Mirage	ParLin	SHMEM
AFAPI	DAPPLE	HPC	MpC	Parmacs	SIMPLE
ALWAN	Data Parallel C	HPF	MOSIX	Parti	Sina
AM	DC++	IMPACT	Modula-P	pC	SISAL.
AMDC	DCE++	ISIS.	Modula-2*	pC++	distributed smalltalk
AppLeS	DDD	JAVAR	Multipol	PCN	SML
Amoeba	DICE.	JADE	MPI	PCP:	SONiC
ARTS	DIPC	Java RMI	MPC++	PH	Split-C.
Athapascan-0b	DOLIB	javaPG	Munin	PEACE	SR
Aurora	DOIME	JavaSpace	Nano-Threads	PCU	Sthreads
Automap	DOSMOS.	JIDL	NESL	PET	Strand.
bb_threads	DRL	Joyce	NetClasses++	PETSc	SUIF.
Blaze	DSM-Threads	Khoros	Nexus	PENNY	Synergy
BSP	Ease .	Karma	Nimrod	Phosphorus	Telegrphos
BlockComm	ECO	KOAN/Fortran-S	NOW	POET.	SuperPascal
C*	Eiffel	LAM	Objective Linda	Polaris	TCGMSG.
"C* in C	Eilean	Lilac	Occam	POOMA	Threads.h++.
C**	Emerald	Linda	Omega	POOL-T	TreadMarks
CarlOS	EPL	JADA	OpenMP	PRESTO	TRAPPER
Cashmere	Excalibur	WWWinda	Orca	P-RIO	uC++
C4	Express	ISETL-Linda	OOF90	Prospero	UNITY
CC++	Falcon	ParLin	P++	Proteus	UC
Chu	Filaments	Eilean	P3L	QPC++	V
Charlotte	FM	P4-Linda	p4-Linda	PVM	ViC*
Charm	FLASH	Glenda	Pablo	PSI	Visifold V-NUS
Charm++	The FORCE	POSYBL	PADE	PSDM	VPE
Cid	Fork	Objective-Linda	PADRE	Quake	Win32 threads
Cilk	Fortran-M	LiPS	Panda	Quark	WinPar
CM-Fortran	FX	Locust	Papers	Quick Threads	WWWinda
Converse	GA	Lparx	AFAPI.	Sage++	XENOOPS
Code	GAMMA	Lucid	Para++	SCANDAL	XPC
COOL	Glenda	Maisie	Paradigm	SAM	Zounds
		Manifold			ZPL

Parallel program environments in the 90's

https://fastcode.org/img/HPEC25_Tasking_and_PyOMP.pdf

In the early days of parallel computing we were obsessed with finding the “right” parallel programming environment

ABCPL	CORRELATE	GLU	Mentat	Parafrese2	
ACE	CPS	GUARD	Legion	Paralation	pC++
ACT++	CRL	HASL	Meta Chaos	Parallel-C++	SCHEDULE
Active messages	CSP	Haskell	Midway	Parallaxis	SciTL
Adl	Cthreads	HPC++	Millipede	ParC	POET
Adsmith	CUMULVS	JAVAR.	CparPar	ParLib++	SDDA.
ADDAP	DAGGER	HORUS	Mirage	ParLin	SHMEM
AFAPI	DAPPLE	HPC	MpC	Parmacs	SIMPLE
ALWAN	Data Parallel C	HPF	MOSIX	Parti	Sina
AM	DC++	IMPACT	Modula-P	pC	SISAL.
AMDC	DCE++	ISIS.	Modula-2*	pC++	distributed smalltalk
AppLeS	DDD	JAVAR	Multipol	PCN	SML
Amoeba	DICE.	JADE	MPI	PCP:	SONiC
ARTS	DIPC	Java RMI	MPC++	PH	Split-C.
Athapascan-0b	DOLIB	javaPG	Munin	PEACE	SR
Aurora	DOIME	JavaSpace	Nano-Threads	PCU	Sthreads
Automap	DOSMOS.	JIDL	NESL	PET	Strand.
bb_threads	DRL	Joyce	NetClasses++	PETSc	SUIF.
Blaze	DSM-Threads	Khoros	Nexus	PENNY	Synergy
BSP	Ease .	Karma	Nimrod	Phosphorus	Telegraphos
BlockComm	ECO	KOAN/Fortran-S	NOW	POET.	SuperPascal
C*	Eiffel	LAM	Objective Linda	Polaris	TCGMSG.
"C* in C	Eilean	Lilac	Occam	POOMA	Threads.h++.
C**	Emerald	Linda	Omega	POOL-T	TreadMarks
CarlOS	EPL	JADA	OpenMP	PRESTO	TRAPPER
Cashmere	Excalibur	WWWinda	Orca	P-RIO	uC++
C4	Express	ISETL-Linda	OOF90	Prospero	UNITY
CC++	Falcon	ParLin	P++	Proteus	UC
Chu	Filaments	Eilean	P3L	QPC++	V
Charlotte	FM	P4-Linda	p4-Linda	PVM	ViC*
Charm	FLASH	Glenda	Pablo	PSI	Visifold V-NUS
Charm++	The FORCE	POSYBL	PADE	PSDM	VPE
Cid	Fork	Objective-Linda	PADRE	Quake	Win32 threads
Cilk	Fortran-M	LiPS	Panda	Quark	WinPar
CM-Fortran	FX	Locust	Papers	Quick Threads	WWWinda
Converse	GA	Lparx	AFAPI.	Sage++	XENOOOPS
Code	GAMMA	Lucid	Para++	SCANDAL	XPC
COOL	Glenda	Maisie	Paradigm	SAM	Zounds
		Manifold			ZPL

Parallel program environments in the 90's

With “choice overload” in mind ...
what did we accomplish with all
these different options for parallel
programming?

Supporting programming models
is a zero-sum game. Time spent
chasing a new/the next
programming model is time NOT
spent making the models we
have actually work well.

This was a headache to
application developers who need
a small number of models that
just work... everywhere and
supported by their systems
vendors.

https://fastcode.org/img/HPEC25_Tasking_and_PyOMP.pdf

The End of the Crisis

- In the **early 90's**, the HPC community was fed up with message passing chaos. Driven largely by application developers, we created **MPI** (version 1.0 released in 1994).
- In the **late 90's**, the HPC community working in the Accelerated Strategic Computing Initiative (ASCI) used their influence over which HPC systems were purchased to “force” vendor’s hands to support a standard for programming shared memory systems.
The result was **OpenMP** (version 1.0 released in 1997).



Portable parallel programming is important for the people who create HPC applications. It took their direct involvement and dedication to create open standards and end parallel programming chaos.

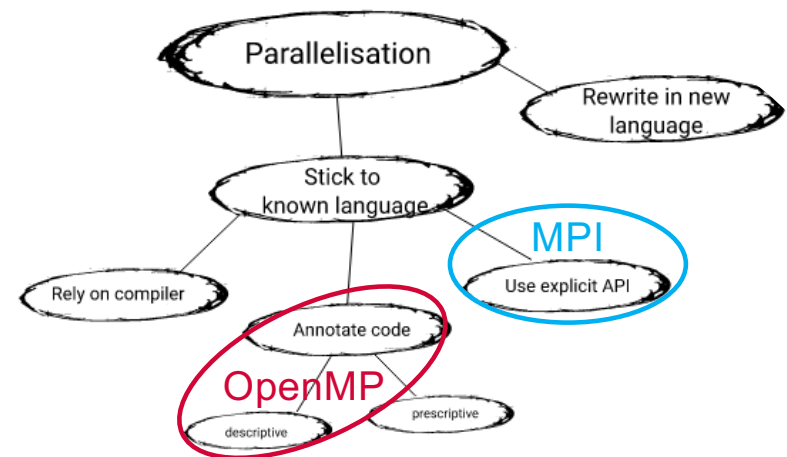
– TIM MATTSON

https://fastcode.org/img/HPEC25_Tasking_and_PyOMP.pdf

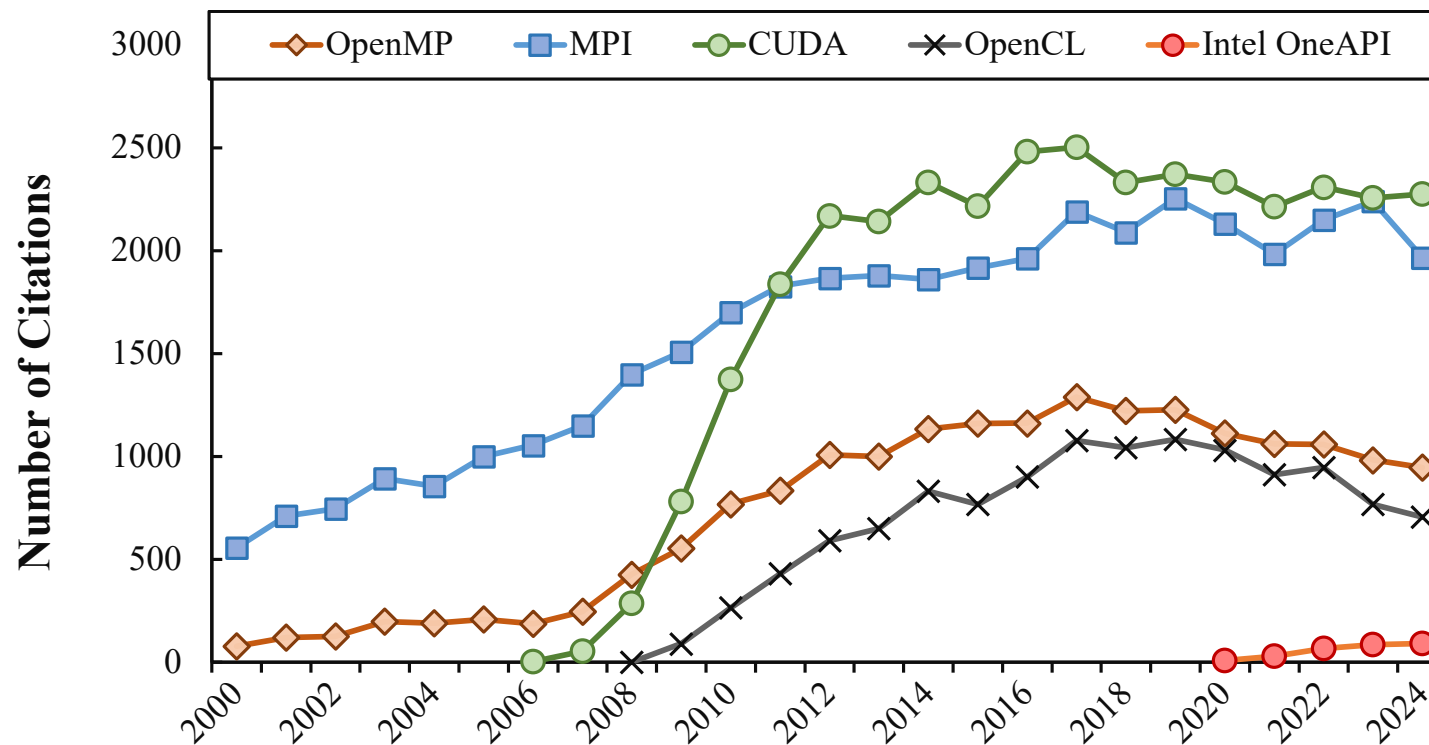
Today's “Standards” for Parallel Programming

Extend Languages (Low-Level Approach)

- Shared-Memory
 - ❑ **OpenMP**, **OpenACC**, Cilk, Intel TBB, Pthreads, Java, Python, C++, Scala (Actors)
- Message Passing
 - ❑ **MPI** (de-facto standard), NCCL
- Accelerator
 - ❑ OpenCL, **CUDA**, HIP, SYCL/oneAPI, Triton
- Partitioned Global Address Space (PGAS)
 - ❑ UPC, Chapel, X10, DASH, HPX
- High-Level AI/ML Parallel Frameworks
 - ❑ PyTorch Distributed, TensorFlow Distributed, JAX pmap/pjit, DeepSpeed, Horovod, Ray (*built atop MPI, NCCL, or custom RPC backends*)
- Domain Specific Languages (DSL)
 - ❑ PATUS, GridTools, DAPHNE (www.daphne-eu.eu), MLIR, XLA
- Domain Specific Architecture (DSA)
 - ❑ Verilog, VHDL, SystemC (not parallel, but crucial for hardware design that enables parallel computing)



Towards **Portable** Programming Models



Citations on the Scopus database of the most used parallel programming models 2000-2024

OpenMP, MPI, CUDA, OpenCL, Sycl/IntelOneAPI, OpenACC: cover the major classes of parallel hardware

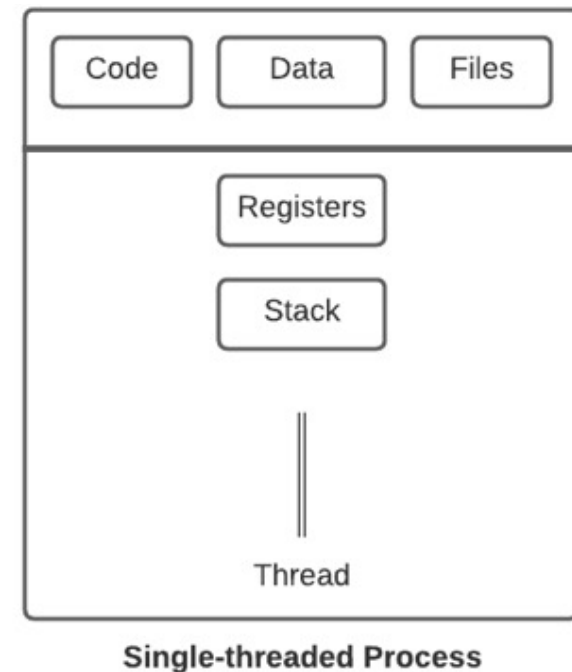
(Software) “Optimizations for any **specific architecture** will have **a short lifespan**.”

– BRUCE HENDRICKSON, IPDPS 2018 KEYNOTE

Sequential Programming: Singleprocess (Singlethread)

Single process and single (default) thread

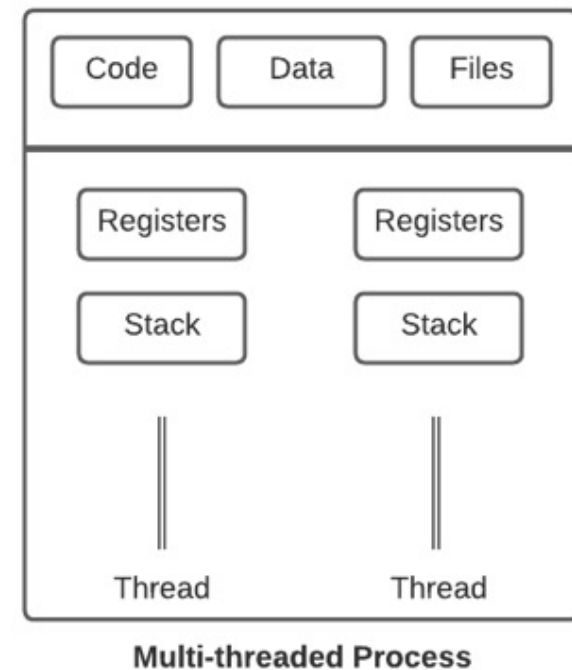
- Thread concept: Independent **flow** of control *within* one process with its own **context** (stack and register set)
- Process has code, data, and opened files used by the default thread
- Thread creation and destruction: **once**
- No intra-process/inter-thread communication necessary
- Example: All operating systems, all programs



Parallel Programming: Singleprocess Multithreaded

Multiple threads (lightweight process model)

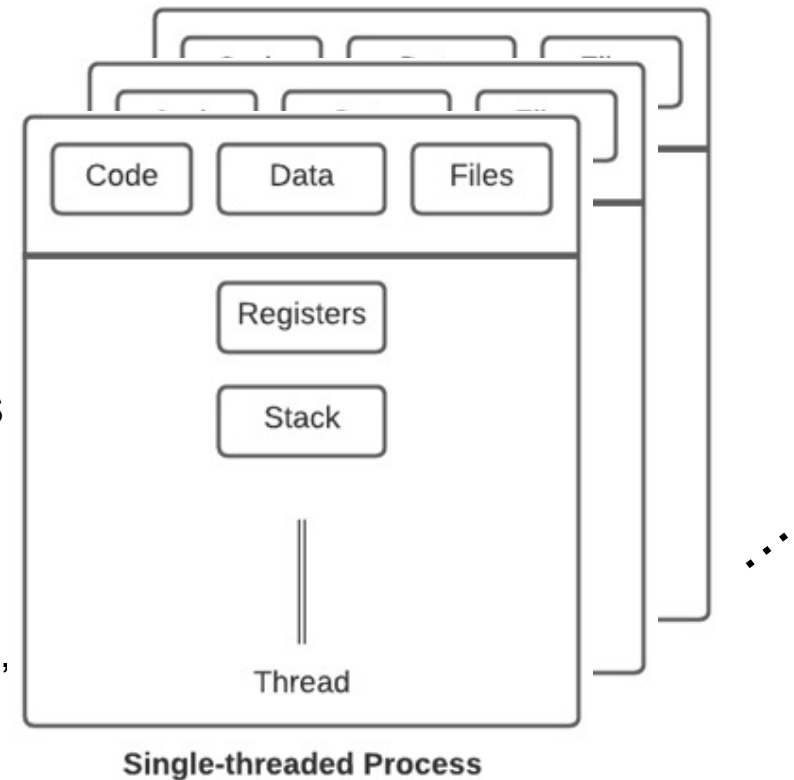
- **Thread** concept: Independent **flow** of control *within* one process with its own **context** (stack and register set)
- Process' code, data, and opened files are **shared** between threads
- Thread creation and destruction: multiple times, *lower overhead* than for multiple processes
- Intra-process (inter-thread) communication techniques supported by OS: **shared memory**
- Example: **OpenMP**, Pthreads



Parallel Programming: Multiprocess Singlethreaded

Multiple processes (heavyweight process model)

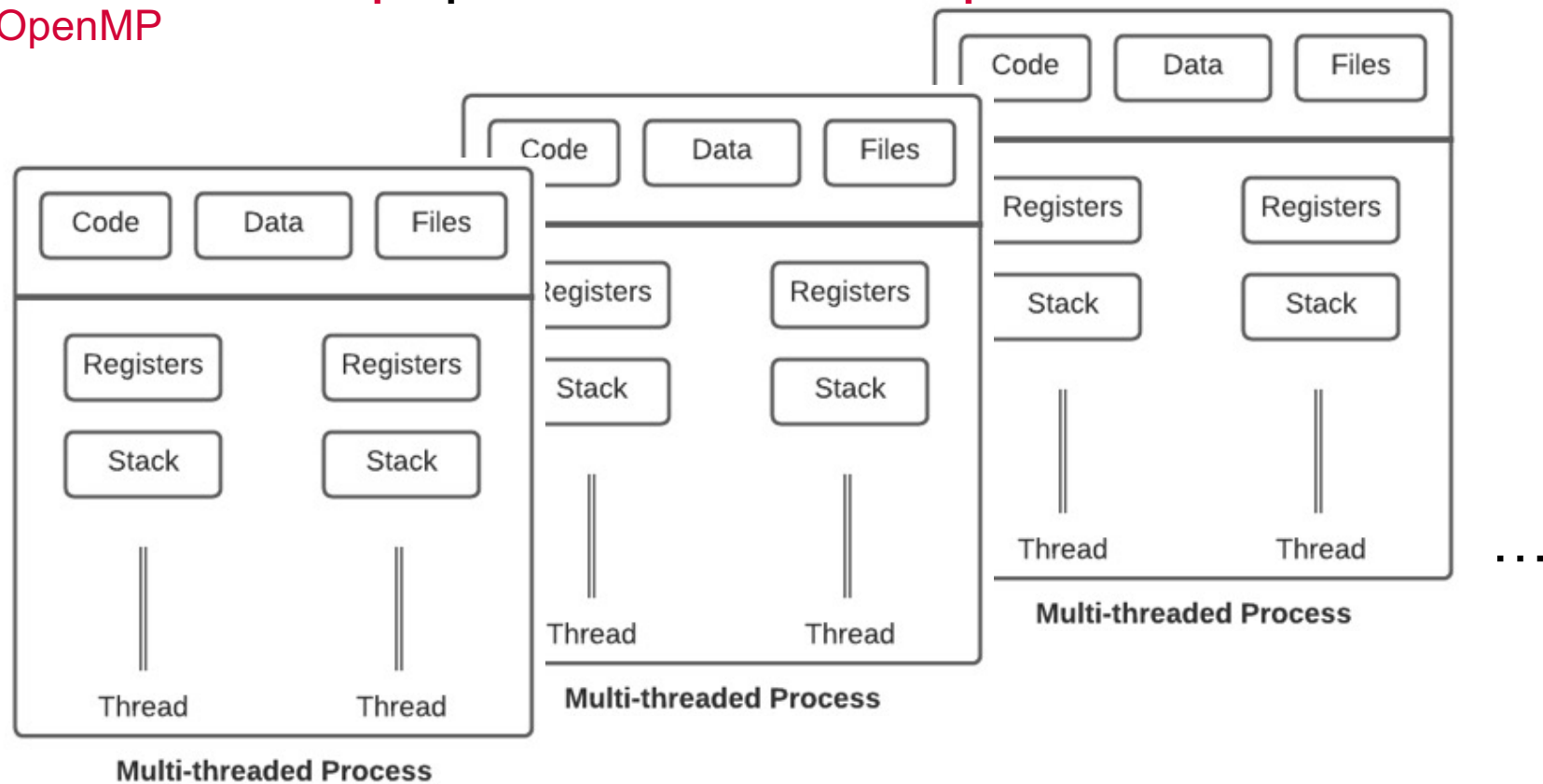
- Traditional UNIX process model
- Processes have different/same code, but different data, and opened files
- Inter-process communication (IPC) techniques supported by OS **shared memory**, sockets over **distributed memory**, file I/O, memory map
- Process creation and destruction: multiple times, **higher overhead** than for multiple threads due to not-shared code, data, files
- Example: **MPI**



Parallel Programming: Multiprocess Multithreaded Programming

Hybrid models involve **multiple processes** each with **multiple threads**

- MPI + OpenMP

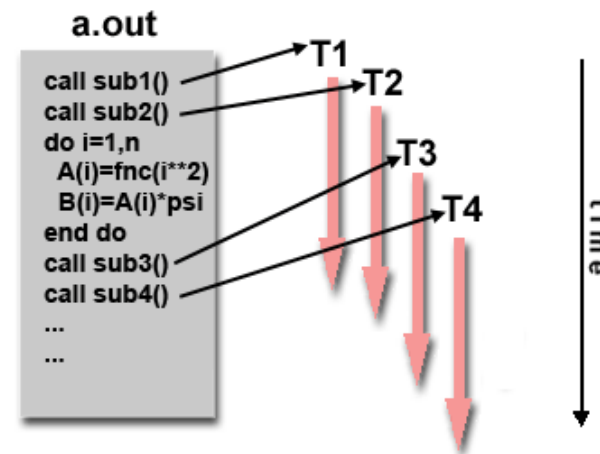


Overview of Chapter 5.3: Parallel Programming

Introduction to Parallel Programming

- a Shared-Memory Programming: OpenMP Introduction
- b Distributed-Memory Programming: MPI Introduction

Parallel programming model 2:
Shared memory (with threads)



What is OpenMP?

Open Multiprocessing

- An **A**pplication **P**rogramming **I**nterface (API) for writing **m**ultithreaded (**MT**) applications
 - A set of **compiler directives** and **library routines** for parallel application programmers
 - + Easy to create **MT** programs in C/C++ and Fortran
 - + Programs execute on **various shared-memory** platforms
 - + Suitable for a **wide range of algorithms** running on multicore nodes and chips, NUMA systems, GPUs, and other such devices attached to a CPU
 - + Very high portability!
 - + Standardizes the last 20+ years practice of **S**ymmetric **M**ultiprocessing (SMP)

⇒ Recall these terms from

Chapter 5.1b. Parallelism: Forms, Levels, Machine Models, Types

About OpenMP



- OpenMP **A**rchitecture **R**evision **B**oard's (ARB) mission is to **standardize** directive-based multi-language (C/C++ & Fortran) **high-level parallelism that is performant, productive and portable**:
www.openmp.org
- **Community** of OpenMP Users, Researchers, Tool Developers and Providers: <http://www.compunity.org/>
- OpenMP **Courses** on the Web
<http://openmp.org/wp/resources/#Tutorials>
https://indico.cern.ch/event/625333/contributions/2587002/attachments/1491364/2318328/intro_par_prog_with_openMP.pdf
- OpenMP **Examples & Reference guide**
www.openmp.org/wp-content/uploads/openmp-examples-6.0.pdf, www.openmp.org/wp-content/uploads/OpenMP-RefGuide-6.0-OMP60SC24-web.pdf
- **Books**
 - R. Chandra *et al.*: *Parallel Programming in OpenMP*. Morgan Kaufmann, 2000.
 - M. Quinn: *Parallel Programming in C with MPI and OpenMP*, McGraw-Hill, 2003.
 - B. Chapman *et al.*: *Using OpenMP: Portable Shared Memory Parallel Programming*, MIT Press, 2007.
 - T. G. Mattson *et al.*: ***The OpenMP Common Core: Making OpenMP Simple Again***, MIT Press, 2019.
https://extremecomputingtraining.anl.gov/wp-content/uploads/sites/96/2019/07/ATPESC_2019_Track-2_2_7-31_830am_Mattson-The-OpenMP_Common_Core.pdf
 - T. Deakin *et al.*: *Programming your GPU with OpenMP*, MIT Press, 2023.
 - ...

1996 Vendors provide similar but different solutions for loop [parallelism](#), causing portability and maintenance problems.
Kuck and Associates, Inc. (KAI) | SGI | Cray | IBM | High Performance Fortran (HPF) | Parallel Computing Forum (PCF)

In spring 7 vendors, Intel, and DOE agree on the spelling of parallel loop and form the OpenMP ARB. By October, version 1.0 of the OpenMP specification for Fortran is released.

1.0

cOMPunity, the group of OpenMP users, is formed, and organizes workshops on OpenMP in North America, Europe, and Asia.

2.0

The OpenMP ARB reaches 15 members of which 5 are supercomputing centers. This mixture of vendors and users is a trademark of OpenMP's cooperative style of operation.

OpenMP releases its first Technical Report that outlines how accelerator and coprocessor devices will be handled.

OpenMP gears toward version 4.1 and 5.0. Topics under discussion include more support for heterogeneous systems, improvements to the tasking model, support for transactional memory, data affinity, and interoperability with other programming models.

Fortran

Minor clarifications.
1.1

Begin discussions about adding task parallelism to OpenMP.

55 pages

76 pages

116 pages

1997 1998 1999 2000 2001 2002 2003 2004 2005 2006 2007 2008 2009 2010 2011 2012 2013 2014

Loop Parallelization

Tasking

Heterogeneity

77 pages

100 pages

242 pages

317 pages

346 pages

538 pages

Unified

3.1
Supports min./max. reductions in C/C++

C/C++

1.0

First hybrid applications with MPI* and OpenMP appear.

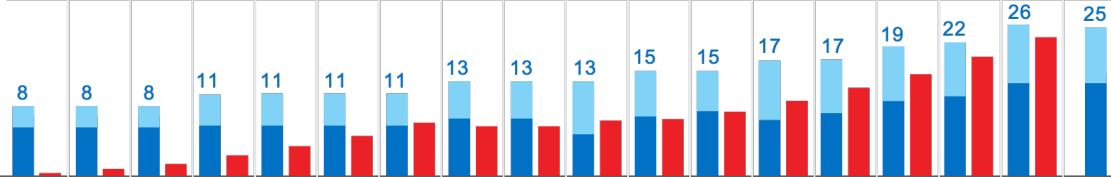
2.0
Merger of Fortran and C/C++ specifications begins.

2.5
Unified C/C++ and Fortran: Bigger than both individual specifications combined. The first International Workshop on OpenMP is held. It becomes a major forum for users to interact with vendors.

3.0
Incorporates task parallelism—a hard problem as OpenMP struggles to maintain its thread-based nature, while accommodating the dynamic nature of tasking.

4.0
Supports accelerator/coprocessor devices, SIMD parallelism, thread affinity, and more. Expands OpenMP beyond its traditional boundaries.

1997 1998 1999 2000 2001 2002 2003 2004 2005 2006 2007 2008 2009 2010 2011 2012 2013 2014



Fall 2025

OpenMP ARB Membership Evolution

Permanent ARB

Auxiliary ARB Members

OpenMP Google Scholar Hits



Origins, Versions, Driving goals

- 2015: OpenMP V4.5 for Fortran and C/C++ (359 p.)
- 2018: OpenMP V5.0 for Fortran and C/C++ (646 p.)
- 2020: OpenMP V5.1 for Fortran and C/C++ (679 p.)
- 2021: OpenMP V5.2 for Fortran and C/C++ (649 p.)
- 2023: OpenMP TR 12 V6.0 preview (772 p.)
- 2024: OpenMP V6.0 for Fortran and C/C++ (934 p.)

www.openmp.org/wp-content/uploads/OpenMP-API-Specification-6-0.pdf

<https://stackoverflow.com/questions/tagged/openmp>

OpenMP Pros and Cons

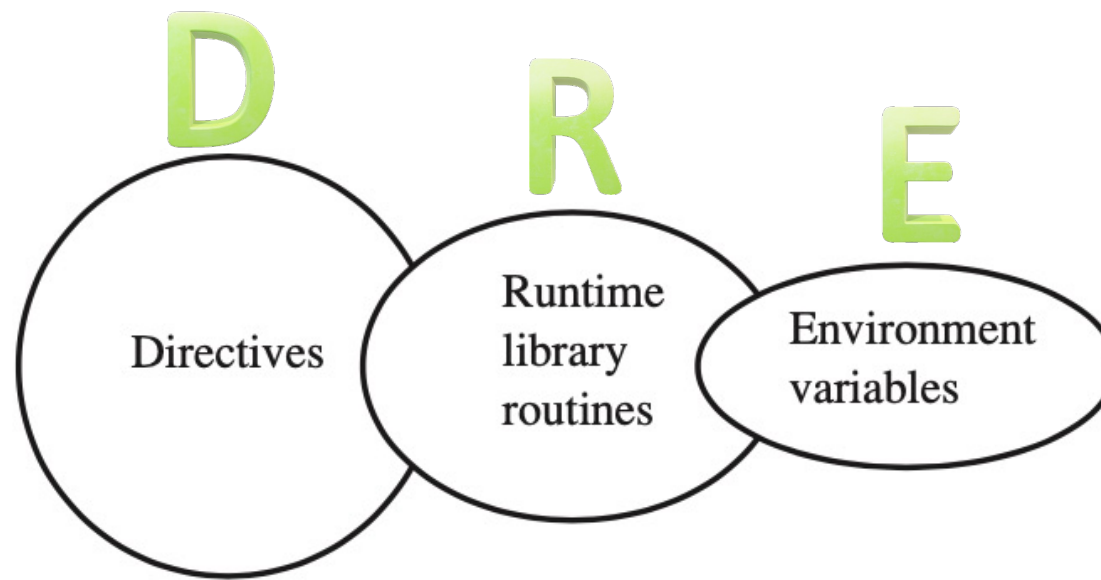
+ Pros

- Widely supported on many × multicore computers
- Portable across OS and compilers (independent of them)
- Now complements MPI in hybrid parallel programming
- Fewer code modifications than using MPI
- OpenMP directives treated as comments if OpenMP unavailable locally
 - Most compilers support OpenMP out of the box
- Incremental adoption by programmers (add directives as needed)
- Supports accelerators and GPUs since OpenMP 4.0 (target directives)

× Cons

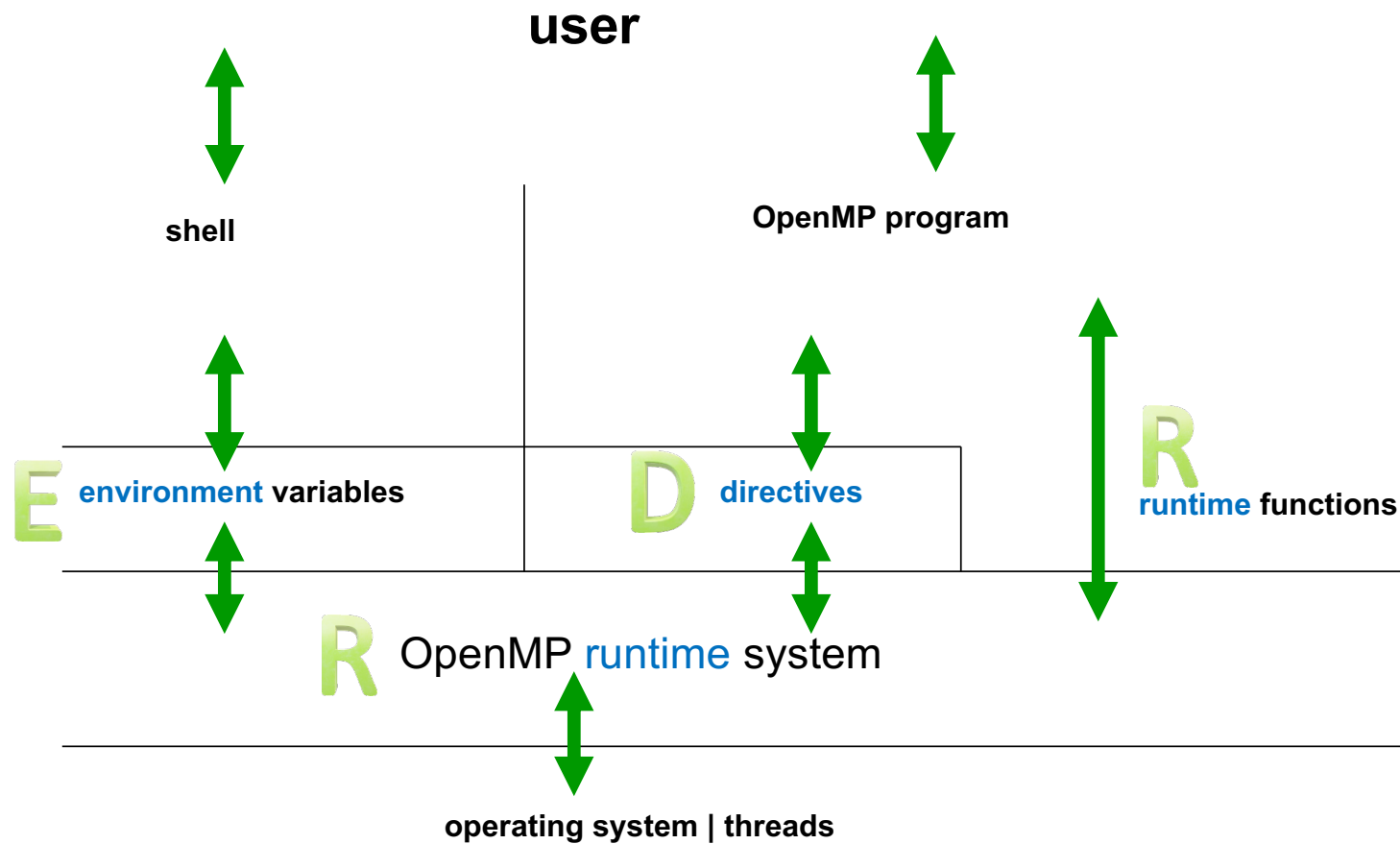
- Limited to shared-memory (local) systems (exception was Intel's ClusterOMP)
- Dependent on compiler implementation and local hardware limits (core count)
- Lower scalability compared to MPI
- Less suited for very fine-grained or for distributed workloads
- May have lower parallel efficiency
 - Rely more on parallelizable loops (data parallelism)
 - **Limited by serial code fraction (Amdahl's Law)**
 - if 50% of code is serial, OpenMP will only ½ time, no matter how many processors

OpenMP API Components



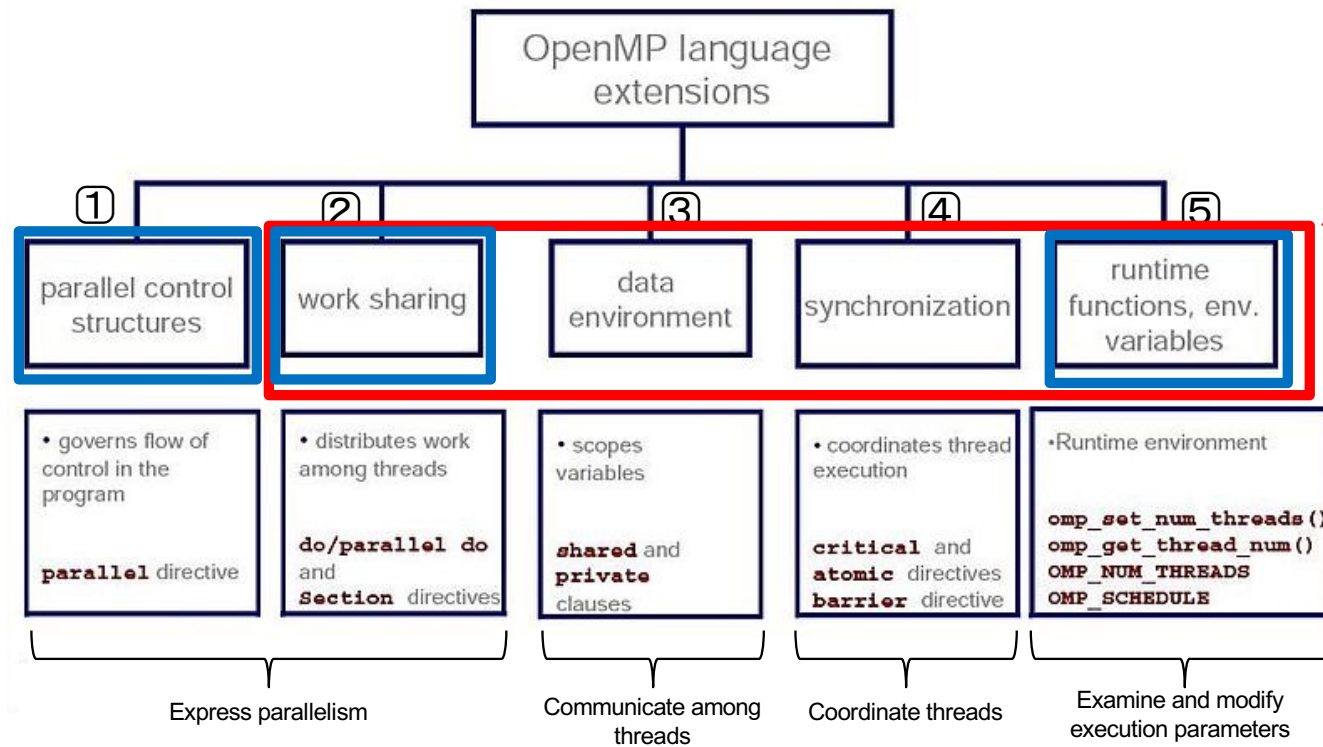
[R. Chandra et al.: Parallel Programming in OpenMP. Morgan Kaufmann, 2000.]

OpenMP Components Diagram



OpenMP Concepts

MORE IN HPC
LECTURE



OpenMP Syntax Reference Guide: C/C++ and Fortran (v6.0, 20 pages)

www.openmp.org/wp-content/uploads/OpenMP-RefGuide-6.0-OMP60SC24-web.pdf

OpenMP: Syntactic Details

- Most constructs in OpenMP are **compiler directives** (or pragmas)

- For C and C++, the pragmas take the form

```
#pragma omp construct [clause [clause]...]
```

- For Fortran, the directives take one of the forms

```
C$OMP construct [clause [clause]...]
```

```
!$OMP construct [clause [clause]...]
```

```
*$OMP construct [clause [clause]...]
```

- All languages require the OpenMP header file

```
#include "omp.h"
```

OpenMP Hello World

① Directives, ⑤ Runtime, Environment

E environment variables

D directives
(special comment lines)

R runtime library function

```
gcc -fopenmp hello-world.c
export OMP_NUM_THREADS=4
./a.out
```



```
#include <stdio.h>
#include <omp.h>
int main(void)
{
    #pragma omp parallel
    printf("Hello World from: %d\n",
        omp_get_thread_num());
}
```

```
Hello world from: 0
Hello world from: 3
Hello world from: 2
Hello world from: 1
```

How Do OpenMP Threads Interact?

- OpenMP is a **shared-memory** programming model
 - ③ Threads communicate by accessing (r|w) **shared variables**
- Unintended sharing of data causes **race conditions**
 - Program's outcome changes as the threads are scheduled & executed in different (non-deterministic) order
- To avoid race conditions
 - ④ Use **synchronization** to protect data access conflicts
 - Change how data is accessed to **minimize the need for synchronization**

OpenMP: Structured Blocks

Most OpenMP constructs (directives) apply to **structured blocks** of code

- A single (or a group of) statements between parentheses **{ }**
 - E.g., one entry point **{** at the top of the statement & one exit point **}** at bottom
- The only “branches” allowed are `exit()` in C/C++

```
if(conv_global()){
    #pragma omp parallel
    {
        int id = omp_get_thread_num();
        int iter = 0; double value;
        do {
            value=do_big_job(id, iter++);
        } while (!conv(value,iter));
        res[id]=value;
    } // ✓ single, natural exit
}
printf(" All done \n");
```

A legal parallel structured block

```
if(conv_global()){
    #pragma omp parallel
    {
        int id = omp_get_thread_num();
        int iter = 0; double value;
        do {
            value=do_big_job(id, iter++);
        } while (!conv(value,iter));
        res[id]=value;
        goto done; // ✗ illegal branch out of parallel region
    }
done: printf(" All done \n");
```

Not a legal structured block

OpenMP: Structured Blocks, Conditional Compilation, Directives with Continued Lines

- Various structured blocks

```
#pragma omp parallel /*++ statem.*/  
{  
    id = omp_thread_num();  
    res(id) = lots_of_work(id);  
}
```

```
#pragma omp for /* single statement */  
for (i=0; i<N; i++)  
{  
    res[i] = big_calc(i);  
    A[i] = B[i] + res[i];  
}
```

- Conditional compilation

```
#ifdef _OPENMP  
    iam=omp_get_thread_num();  
#endif  
// allows code to run seamlessly without  
// -fopenmp flag
```

- Continued lines

```
/* OpenMP directive */  
#pragma omp directive [clause ..]  
  
/* OpenMP directive with  
continuation line */  
#pragma omp directive clause \  
    clause ...
```

OpenMP: Sequential Region and ① Parallel Region

```
#include "omp.h"
void main()
{
    printf("inside parallel region? %d\n", omp_in_parallel());
    printf("number of available processors? %d\n", omp_get_num_procs());
    printf("maximum number of threads? %d\n", omp_get_max_threads());
    omp_set_num_threads (omp_get_max_threads() );
    #pragma omp parallel
    {
        printf("inside parallel region %d\n", omp_in_parallel());
        printf("number of threads in the team %d\n", omp_get_num_threads());
        printf("my thread id %d\n", omp_get_thread_num());
    }
}
```

Sequential region

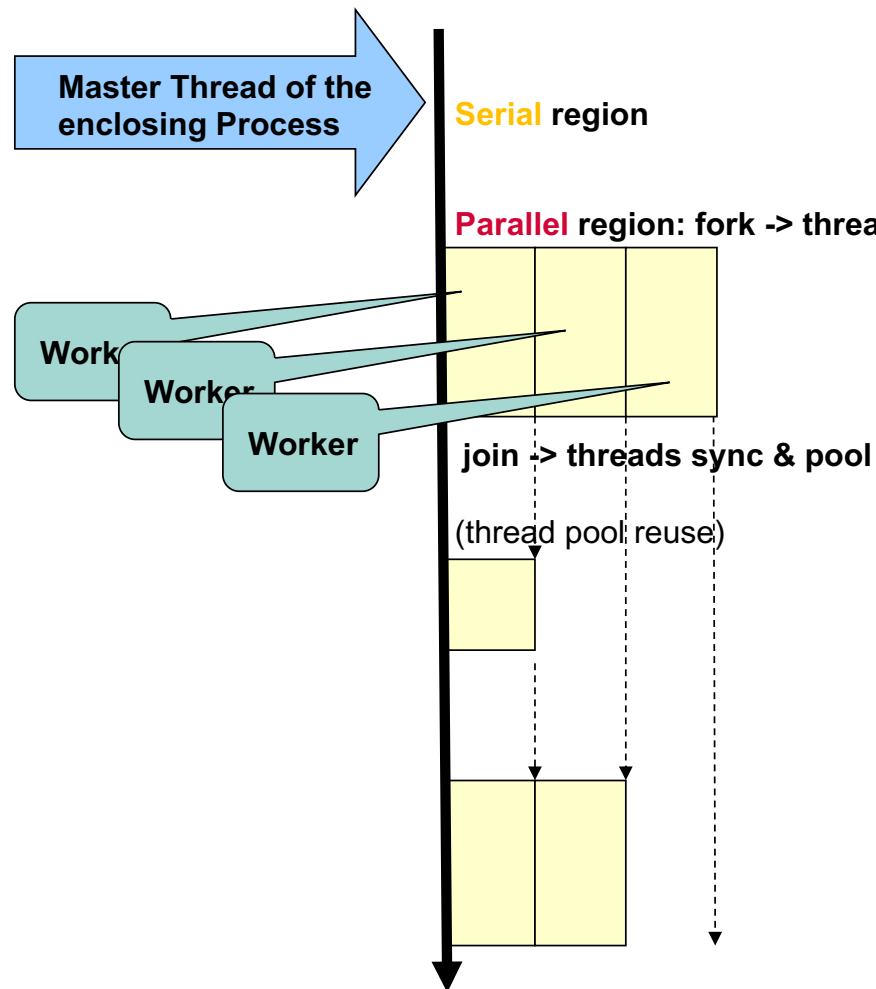
Parallel region

What type of OpenMP regions do we have here and where?

OpenMP: ⑤ Runtime Library (RTL) Functions

Functions	Serial region	Parallel region
void <code>omp_set_num_threads (int)</code>	Set # threads to use in a team	No effect
int <code>omp_get_num_threads (void)</code>	1	Return # threads
int <code>omp_get_max_threads (void)</code>	Return max # threads (OMP_NUM_THREADS)	
int <code>omp_get_thread_num (void)</code>	0	Return thread id 0 ... #threads-1
int <code>omp_get_num_procs (void)</code>	Return # CPUs	
void <code>omp_set_dynamic (int)</code>	Control dynamic adjustment of # threads	No effect
int <code>omp_get_dynamic(void)</code>	TRUE: if dynamic thread adjustment enabled FALSE: Otherwise	
int <code>omp_in_parallel (void)</code>	FALSE	TRUE

OpenMP: ① Parallel Regions & Fork-Join Parallelism



The OpenMP program initially starts as a serial program: single threaded...

Upon entering the first **parallel region**, worker threads are started via **fork** (either created/reused) by OpenMP runtime. Together with the master, they form a (thread) team.

Between the parallel regions (aka **join**), the worker threads are put to “sleep” ^{zzz} into a thread pool (since OpenMP 3.0)...

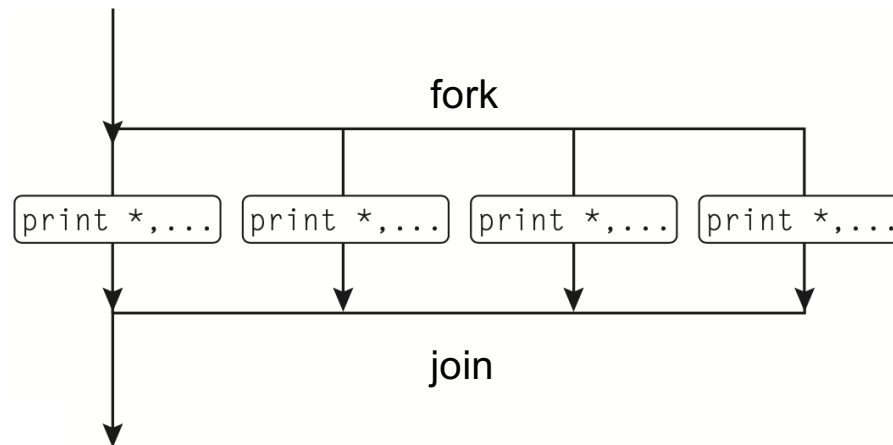
... and (re)**awakened** ⌚ when needed at entering the other **parallel regions**.

```
// fork → parallel → join
#pragma omp parallel
{
    work(id);
}
```

OpenMP: ① Parallel Regions & Fork-Join Parallelism Example

A simple parallel region.

```
...  
!$omp parallel  
    print *, 'Hello world'  
!$omp end parallel  
...
```



Runtime execution model for a parallel region.

[R. Chandra et al.: Parallel Programming in OpenMP. Morgan Kaufmann, 2000.]

OpenMP: ① Parallel Regions & Fork-Join Parallelism Example (cont.)

- Create a parallel region and forked 4 threads

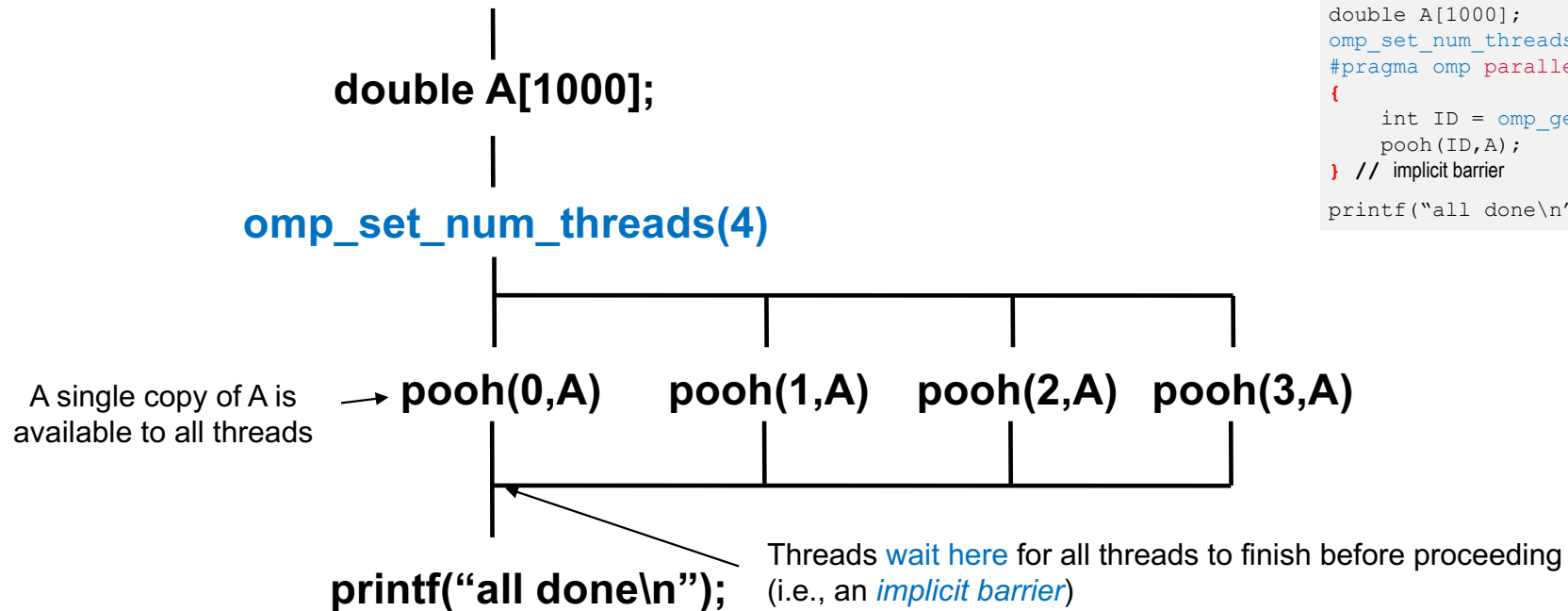
```
double A[1000];  
omp_set_num_threads(4);  
#pragma omp parallel  
{  
    int ID = omp_get_thread_num();  
    pooh(ID, A);  
} // implicit barrier  
printf("all done\n");
```

Runtime library function to request a certain number of threads

Runtime library function that returns thread ID

- Each thread executes independently a **copy of the code** within the structured block **{ }** of the enclosing parallel region
 - Each thread with their ID = 0 to 3 calls `pooh(ID, A)`

OpenMP: ① Parallel Regions & Fork-Join Parallelism Example (cont.)

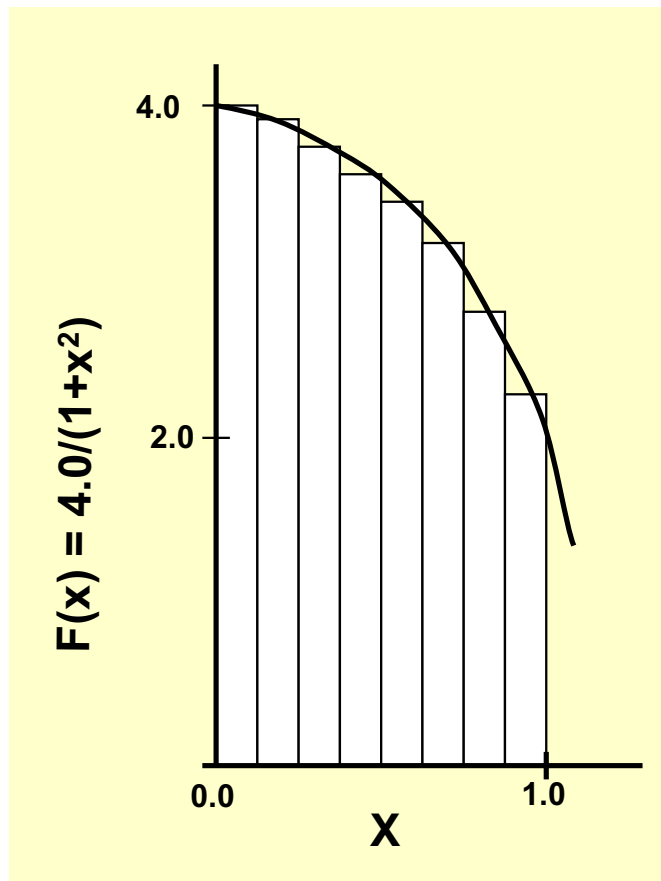


```
double A[1000];  
omp_set_num_threads(4);  
#pragma omp parallel  
{  
    int ID = omp_get_thread_num();  
    pooh(ID, A);  
} // implicit barrier  
printf("all done\n");
```

- Each thread executes independently a **copy of the code** within the structured block `{ }` of the enclosing parallel region
 - Each thread with their ID = 0 to 3 calls `pooh (ID, A)`

OpenMP: ① Parallel Regions & Fork-Join Parallelism

Another Example: Calculating π



Mathematically, we calculate π as:

$$\int_0^1 \frac{4.0}{(1+x^2)} dx = \pi.$$

We can approximate the integral as a sum of rectangles:

$$\sum_{i=0}^N F(x_i) \Delta x \approx \pi$$

Each rectangle has width Δx and height $F(x_i) = 4.0/(1+x^2)$ at the *middle* of interval i .

OpenMP: ① Parallel Regions & Fork-Join Parallelism

Sequential Program for Calculating π

```
static long num_steps = 100000; double step;

void main () {
    int i; double x, pi, sum = 0.0;
    step = 1.0/(double) num_steps; // step size ( $\Delta x$ )

    for (i=0; i<num_steps; i++) {
        x = (i+0.5)*step;
        sum = sum + 4.0/(1.0+x*x); //  $\sum F(x_i)$ 
    }

    pi = sum * step; //  $\sum F(x_i)\Delta x$ 
    printf("Sequential approximation of pi = %.15f\n", pi);
} //end main;
```

OpenMP: ① Parallel Regions & Fork-Join Parallelism

Employing SPMD Programming Model for Calculating π – Take 1

Need `i_local` to avoid race on `i` or make it `private(i)` for manually partitioned loops

```
#include <omp.h>
static long num_steps = 100000; double step;
#define NUM_THREADS 16
void main () {
    int i; double pi, ... sum[NUM_THREADS]=0 ...
    step = 1.0/(double)num_steps;
    omp_set_num_threads(NUM_THREADS);
    #pragma omp parallel {
        double x; int id, i_local;
        id = omp_get_thread_num();
        for (i_local=id; i_local<num_steps; i_local=i+NUM_THREADS) {
            x = (i_local+0.5)*step;
            sum[id] = sum[id] + 4.0/(1.0+x*x);
        } //end parallel region (implicit barrier)
        for (i=0, pi=0.0; i<NUM_THREADS;i++)
            pi = pi + (sum[i] * step);
        printf("OpenMP approximation of pi = %.15f (threads=%d, steps=%ld)\n",
            pi, NUM_THREADS, num_steps);
    } //end main
```

SPMD Model

Each thread runs the same code, with the thread ID selecting any thread-specific behavior to execute.

Parallel region

Manual work sharing

Sequential region to reduce sum

OpenMP: ① Parallel Regions & ② Work Sharing

How OpenMP is Typically Used

OpenMP was designed for and is typically used to parallelize loops

1. Find the **most time-consuming** loop(s)
2. **Split it (them) up** (data parallelism) automatically between threads

```
void main()
{
    double Res[1000];

    for (int i=0; i<1000; i++)
    {
        do_huge_comp(Res[i]);
    }
}
```

Sequential Program

```
#include "omp.h"
void main()
{
    double Res[1000];
    #pragma omp parallel for
    for (int i=0; i<1000; i++)
    {
        do_huge_comp(Res[i]);
    }
}
```

OpenMP Parallel Program

i is automatically private by default as this is used in a omp parallel for

OpenMP: ① Parallel Regions & ② Work Sharing – Principle

Apply **Data Parallel** Programming Model (4) to Vector Sum

Process and work sharing loop

```
#pragma omp parallel for
for (i=0; i<1000; i++)
    a[i]=b[i]+c[i]
```

Threads automatically get

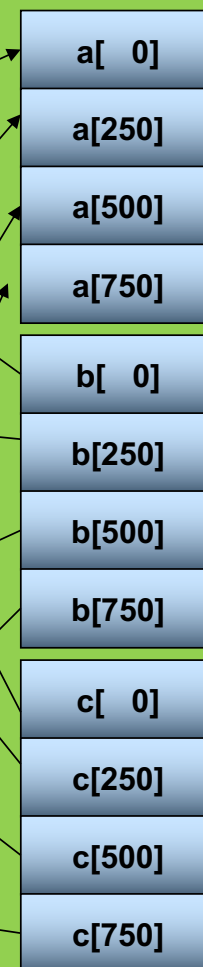
```
for (i= 0; i<250; i++)
    a[i]=b[i]+c[i]
```

```
for (i=250; i<500; i++)
    a[i]=b[i]+c[i]
```

```
for (i=500; i<750; i++)
    a[i]=b[i]+c[i]
```

```
for (i=750; i<1000; i++)
    a[i]=b[i]+c[i]
```

Shared
Memory



OpenMP: ② Work Sharing

Apply SPMD Programming Model (6) to Vector Sum

Sequential code

```
for(i=0; i<N; i++)    { a[i] = a[i] + b[i]; }
```

OpenMP parallel region

+ *manual work sharing*

```
#pragma omp parallel
{
    int i, id, Nthrds, istart, iend;
    id = omp_get_thread_num();
    Nthrds = omp_get_num_threads();
    istart = id * N / Nthrds;
    iend = (id+1) * N / Nthrds;
    for(i=istart; i<iend; i++)
    { a[i] = a[i] + b[i]; }
}
```

or

OpenMP parallel region

+ **implicit work sharing**
with the “for” construct

```
#pragma omp parallel
#pragma omp for schedule(static)
for(i=0; i<N; i++)
{
    a[i] = a[i] + b[i];
}
```

OpenMP: ① Parallel Regions vs. ② Work Sharing Constructs (for Data Parallelism and SPMD)

```
#pragma omp parallel  
  
for (int i=0;i<N;i++)  
{  
    NEED_STUFF(i);  
}
```

Each thread executes **entire loop**, all threads iterate over the same “i”, **shared by default** because the for loop is inside a `parallel` region but not inside a `parallel for` region

No work sharing, just parallelism

unless we manually express the work to be conducted by each thread (e.g., `istart`, `iend`) → steps 3 (mapping) and 4 (programming)

Risk of race condition if `i` used beyond the loop

```
#pragma omp parallel  
#pragma omp for [nowait]  
for (i=0;i<N;i++)  
{  
    NEED_STUFF(i);  
} //implicit barrier
```

The “**omp for**” **work sharing construct** tells OpenMP to split up iterations among the threads in a team. `i` is **automatically private** according to the OpenMP standard and each thread gets its own copy.

Each thread executes **only** a subset of iterations (e.g., block or cyclic scheduling).

By default, an *implicit barrier* occurs at the end of the “omp for”, unless `nowait` is specified.

OpenMP: ① Parallel Regions and ② Work Sharing

Apply SPMD Programming Model for Calculating π – Take 2

```
#include <omp.h>
static long num_steps = 100000; double step;
#define NUM_THREADS 16
void main (){
    int i; double pi, ... sum[NUM_THREADS]=0 ...
    step = 1.0/(double) num_steps;
    omp_set_num_threads(NUM_THREADS);
    #pragma omp parallel {
        double x; int id, i;
        id = omp_get_thread_num();
        #pragma omp for
        for (i=0; i<num_steps; i++){
            x = (i+0.5)*step;
            sum[id] = sum[id] + 4.0/(1.0+x*x); } //end parallel for
    } //end parallel region (implicit barrier)
    for (i=0, pi=0.0; i<NUM_THREADS;i++)
        pi = pi + sum[i] * step;
    } //end main
```

The remaining code (per-thread partial sums and final combine) is still necessary.

Parallel region

Automatic work sharing

Sequential region to reduce sum

OpenMP: Compound Construct

① Parallel Regions + ② Work Sharing

Tip/shortcut: Place “**parallel**” and work sharing “**for**” on the same line.

```
double res[MAX];
int i;
#pragma omp parallel
{
    #pragma omp for
    for (i=0; i< MAX; i++)
    {
        res[i] = huge();
    }
}
```

```
double res[MAX];
int i;
#pragma omp parallel for
    for (i=0; i< MAX; i++)
    {
        res[i] = huge();
    }
```

These are equivalent

OpenMP in the Wild: Beyond Supercomputers

Everyday Software and AI Tools Accelerated by OpenMP

Creative & Media Software

- **Blender** (3D modeling/rendering) – Compositor and part of Cycles renderer use OpenMP (pixel shading...).
- **GIMP** (GNU image manipulation program) – OpenMP parallelizes image filters and pixel operations
- **FFmpeg** – Video filters (scaling, color conversion, motion estimation) have optional OpenMP acceleration
- **Audacity** – Some DSP and FFT code paths can be multithreaded with OpenMP on CPUs

Science & Bioinformatics

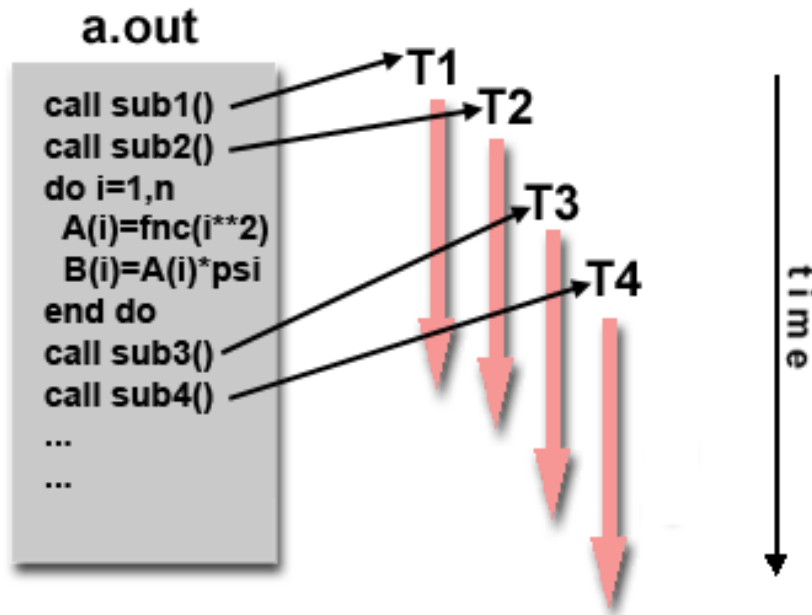
- **Tensorflow/Pytorch (CPU backend)** – CPU kernels (matmul) are OpenMP parallelized in the Eigen/MKL-DNN (oneDNN) backends
- **LAMMPS, GROMACS** hybrid – short-range force kernels use OpenMP for shmem acceleration / MPI process

Data, AI, and Libraries

- **NumPy** – numpy.dot, linalg and fft calls are OpenMP-parallelized via MKL/OpenBLAS (OpenMP backend).
- **XGBoost. LightGBM** – Tree construction and training parallelized with OpenMP across CPU cores.
- **OpenCV** – Image filtering and transforms (convolution, resize, morphology) parallelized with OpenMP

If your laptop fan spins up, OpenMP might be to blame.

What Have We Learned So Far?



Parallel programming model 2:
Shared-Memory (with threads)

Introduction to Parallel Programming

- a Shared-Memory Programming: OpenMP Intro
- b Distributed-Memory Programming: MPI Intro

Summary

- Obsession with the “right” parallel model ended early 90’s with: **OpenMP**, MPI, CUDA, HIP, Sycl, OpenACC
- Extend languages: low-level approach to parallel programming, with pros/cons
- **OpenMP** has both pros and cons
- **D**irectives, **R**untime, **E**nvironment
 - **D**: Parallel regions
 - **D**: Work sharing
 - **R**untime library functions
 - **E**nvironment variables